

Développement mobile

Software Development Kit (SDK) Flutter

Généralité sur le développement mobile

Dr Ibrahima GAYE
ibrahima2.gaye@uadb.edu.sn

Universite Alioune DIOP (UAD) de Bambey

27 octobre 2021



Références

- ▶ <https://flutter.dev/docs/get-started/install/windows>
- ▶ <https://www.udemy.com/course/flutter-dart-creez-des-applications-pour-ios-et-android/>
- ▶ <https://www.youtube.com/watch?v=LgbB3e7xuiA>
- ▶ <https://flutter-love.com/introduction-a-flutter/>

Les pré-requis

- ▶ **Algorithmique** : la base de la programmation ;
- ▶ **JAVA** : les applications sous android sous essentiellement en Java (aussi en **Kotlin** depuis 2017) ;
- ▶ **SQL** : connaître un minimum de SQL pour les requêtes ;
- ▶ **Une culture en informatique** : plateformes de développement comme Eclipse, savoir installer des plugins, etc.

Historique des plateformes mobiles

Quelques plate-formes

Plateformes de développement mobile

Téléphone portable

- ▶ Norme GSM créée en 1982
- ▶ Premier téléphone portable fabriqué en 1983 par Motorola
- ▶ 1986, 1er réseau en France : Radiocom 2000
- ▶ 1992, France Telecom lance Itineris

Les assistants numériques personnels (PDA)

- ▶ Années 1990
- ▶ Agenda, carnet d'adresses, bloc-notes
- ▶ Système d'exploitation
- ▶ Synchronisation des données avec un PC

Les smartphones

- ▶ SMARTPHONE = PDA + téléphone portable
- ▶ Lancé en 2001 Sagem
- ▶ L'essor commence en 2005 avec
 - ▶ la sortie de l'iPhone ;
 - ▶ Le rachat de la startup Android, par Google.
- ▶ GPS et Wifi : téléphone + connexion Internet
- ▶ Nombreuses fonctionnalités
- ▶ Système d'exploitation
- ▶ Se comporte comme un ordinateur portable
- ▶ Téléchargement d'applications

Historique des plateformes mobiles (4)

Evolution des téléphones



Les magasins de d'applications

- ▶ Plateformes de téléchargement (gratuit ou payant) de logiciels pour smartphones.
- ▶ Appstore d'Apple, Google Play, Windows Phone, etc.
- ▶ Les éditeurs d'applications sont obligés de déposer leurs logiciels sur ces plateformes, afin qu'elles soient disponibles
- ▶ Modèles économiques
 - ▶ Application payante
 - ▶ Publicité
 - ▶ Contenu payant

Les systèmes d'exploitation mobiles

- ▶ Ancienne génération
 - ▶ Symbian (Nokia)
 - ▶ BlackBerry OS
 - ▶ Windows Mobile
- ▶ OS "tactiles"
 - ▶ IOS
 - ▶ dérivé de Mac OS X
 - ▶ Plusieurs applications téléchargeables sur l'Appstore
 - ▶ Android
 - ▶ Basé sur un noyau Linux
 - ▶ Open source
 - ▶ Intègre les services Google
 - ▶ Windows Phone 7
 - ▶ BlackBerry

Quelques plate-formes

Architecture Android



Source : https://user.oc-static.com/files/318001_319000/318598.jpg

Architecture Android

- ▶ Applications
 - ▶ Les applications au niveau de la couche supérieure de l'architecture
 - ▶ Les applications natives et tierces telles que les contacts, email, la musique, la galerie, l'horloge, les jeux, etc.
 - ▶ La couche d'application s'exécute dans l'exécution d'Android à l'aide des classes et des services disponibles à partir de l'Application Framework.

Architecture Android

- ▶ Application Framework
 - ▶ Cette couche fournit les classes utilisées pour créer une application Android,
 - ▶ elle fournit également une abstraction générique pour l'accès au matériel et gère l'interface utilisateur et les ressources de l'application.
 - ▶ Elle comprend aussi des services (qui sont autorisés aux développeurs) tels que service de téléphonie, services de localisation, gestionnaire de notification, service NFC, système de visualisation, etc. que nous pouvons utiliser pour le développement d'applications.

Architecture Android

- ▶ Application Framework
 - ▶ Cette couche comprend les services clés suivants :
 - ▶ Activity Manager : Contrôle tous les aspects du cycle de vie de l'application et de la pile d'activités (Activity stack).
 - ▶ Content Providers : Permet aux applications de publier et de partager des données avec d'autres applications.
 - ▶ Resource Manager : Fournit un accès à des ressources incorporées non codées telles que des chaînes, des paramètres de couleur et des dispositions d'interface utilisateur (Strings, colors, layouts).
 - ▶ Notifications Manager : Permet aux applications d'afficher des alertes et des notifications à l'utilisateur.
 - ▶ View System : Ensemble extensible de vues utilisé pour créer des interfaces utilisateur d'application.

Architecture Android

- ▶ Android Runtime
 - ▶ Cette couche fournit un composant clé appelé **Dalvik Virtual Machine**, une sorte de machine virtuelle Java spécialement conçue et optimisée pour Android ; Responsable de l'exécution des applications Android
 - ▶ Cette couche contient des composants tels que les core libraries(bibliothèques de base) et la machine virtuelle Dalvik.
 - ▶ Elle est le moteur qui alimente nos applications avec les bibliothèques et constitue la base d'Application Framework.

Architecture Android

► Android Runtime

- Dalvik Virtual Machine (DVM) est une machine virtuelle basée sur des registres, Il est spécialement conçu et optimisé pour Android afin de garantir qu'un appareil puisse exécuter plusieurs instances efficacement.
- DVM permet à chaque application Android de s'exécuter dans son propre processus, avec sa propre instance de la machine virtuelle Dalvik.
- Les core libraries nous permettront d'implémenter des applications Android utilisant le langage de programmation JAVA standard.

Architecture Android

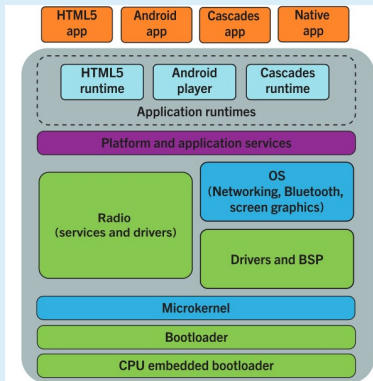
- ▶ Platform Libraries
 - ▶ Incluent les libraries C/C++, Java telles que SSL, libc, Graphics, SQLite, Webkit, Media, Surface Manager, OpenGL, etc pour la prise en charge du développement Android
 - ▶ Détails de certaines libraries
 - ▶ Media library pour lire et enregistrer des formats audio et vidéo
 - ▶ Surface Manager pour fournir une gestion d'affichage
 - ▶ SGL et OpenGL Graphics pour les graphiques 2D et 3D
 - ▶ SQLite est destiné au support de base de données et FreeType au support de polices (font)
 - ▶ Web-Kit pour la prise en charge des navigateurs Web et SSL pour la sécurité Internet.

Architecture Android

- ▶ Linux Kernel
 - ▶ Composant principal d'android
 - ▶ Gère les fonctions de son système d'exploitation aux téléphones mobiles
 - ▶ Couche inférieure (cœur de l'architecture Android)
 - ▶ Gère tous les drivers : affichage, caméra, Bluetooth, audio, mémoire, Accès aux périphériques Android lors de l'exécution, etc.

Quelques plate-formes

Architecture BlackBerry



Source : Security Overview BlackBerry June 2015

Architecture Blackberry

- ▶ CPU embedded bootloader
 - ▶ Vérifie la signature numérique du code du bootloader avant qu'il ne puisse s'exécuter .
- ▶ Microkernel
 - ▶ Quantité minimale de logiciels dont le système d'exploitation a besoin pour fonctionner.
- ▶ Radio
 - ▶ Elle comprend :
 - ▶ les pilotes
 - ▶ les piles
 - ▶ les services requis pour prendre en charge les sous-systèmes radio pour la voix, les données et d'autres services

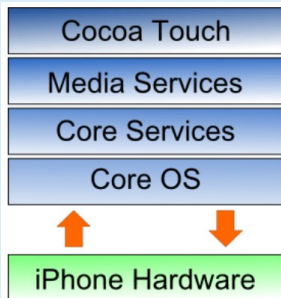
Architecture Blackberry

- ▶ Drivers and BSP incluent les pilotes et la carte (fonctionnement logique) pour prendre en charge le matériel de l'appareil
- ▶ OS : Les processus du OS qui existent en dehors du kernel
- ▶ Platform and application services incluent :
 - ▶ la sécurité
 - ▶ installation des logiciels
 - ▶ services arrière plan des logiciels
 - ▶ services multimédia
 - ▶ accès aux composants et services protégés
 - ▶ etc.

Architecture Blackberry

- ▶ Application runtimes incluent :
 - ▶ les machines virtuelles,
 - ▶ les libraries et services,
 - ▶ mappage des couches
- ▶ App : gère Les applications préchargées, installées par l'utilisateur ou déployées par votre organisation

Architecture Iphone



Source :

https://www.techotopia.com/index.php/The_iPhone_OS_Architecture_and_Frameworks

Now that we have identified the various layers that comprise the iPhone OS we can now look in more detail at the services provided by each layer and the corresponding frameworks that make those services available to us as app developers.

Architecture Iphone

- ▶ Cocoa Touch
 - ▶ Écrite principalement en Objective-C et basée sur l'API Cocoa
 - ▶ Se situe au sommet de la pile de l'OS et contient les frameworks les plus couramment utilisés par les développeurs d'applications iPhone.

Architecture Iphone

- ▶ Media layer
 - ▶ fournit à l'OS de l'iPhone des capacités audio, vidéo, d'animation et graphiques.
 - ▶ fournit aux autres couches la pile iPhone OS,
 - ▶ elle comprend un certain nombre de frameworks qui peuvent être utilisés lors du développement d'applications iPhone.

Architecture Iphone

- ▶ Core Data Framework
 - ▶ Facilite la modélisation et le stockage des données dans les applications basées sur le MVC.
 - ▶ Réduit considérablement la quantité de code qui doit être écrit pour effectuer des tâches courantes lors de l'utilisation de données structurées dans une application.

Architecture Iphone

- ▶ Core OS
 - ▶ Couche inférieure de la pile iPhone OS et se trouve directement au-dessus du matériel de l'appareil.
 - ▶ Fournit la mise en réseau ; l'accès aux accessoires externes ; les services fondamentaux habituels du système d'exploitation tels que la gestion de la mémoire, la gestion du système de fichiers et les threads.

Quelle plateforme mobile

Il existe, à l'heure actuelle, plusieurs plateformes pour développer une application mobile dont les plus utilisées sont : **native**, **hybride** et **cross-platform**

► Quelques critères de choix :

- Destiner a quel clientèle ;
- Performance technique pour supporter l'application ;
- Comment l'application est architecturée ;
- Application simple qui n'a pas une charge de calcul importante ;
- Destiner a une large public avec de faible revenus ;
- Application qui est destinée a du haut de gamme ou a une sous couche de la population

Quelles plateformes mobile

- ▶ Partir sur un développement natif (Objective C ou swift, java, C#, kotlin, etc.)
 - ▶ **Cross-platform** : Des solutions telles que **Xamarin**, **Flutter** permettent de construire des applications iOS, Android et Windows mobile dans un environnement de développement Microsoft (C#)
 - ▶ **hybrides** : Des technologies de type **Phonegap** ; **Cordova** ; **IONIC** ; **etc.** permettent d'embarquer une web view dans une application native.
 - ▶ Opter pour un **développement web** item etc.

Plateformes natives

► Avantages

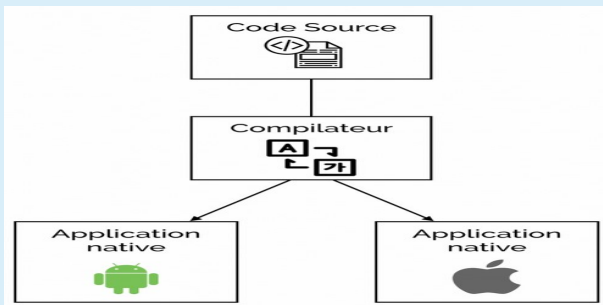
- Chaque système d'exploitation a sa version native
- Possibilité de faire appel aux fonctionnalités de l'appareil utilisé (telle que la camera, la géolocalisation, etc.) et ainsi les exploiter de manière infinie
- Profiter d'une visibilité sur les stores d'applications et optimisation du référencement
- Accès aux APIs natives
- Plus simple à développer et plus d'outils à disposition et de support en cas de problème
- Intégration à la facturation des stores si l'objectif est de rentabiliser le produit (Apps payantes)
- Facilité à se conformer aux ergonomies spécifiques de chaque système d'exploitation afin d'avoir la meilleure expérience utilisateur possible

Plateformes natives

- ▶ Inconvénients
 - ▶ Le coût de développement est important car il faut développer une version spécifique pour chaque système d'exploitation
 - ▶ Les performances sont très souvent meilleures en natif
 - ▶ Toutes les mises à jour nécessiteront un coût de développement supplémentaire multiplié par le nombre de plate-formes utilisées

Cross-Platform

- ▶ Développer pour un environnement et c'est cet environnement qui sera déployé dans plusieurs plateformes : Ex : JavaScript, Dart, etc.



Cross-Platform

- ▶ Avantages
 - ▶ Chaque système d'exploitation a sa version native
 - ▶ Profiter d'une visibilité sur les stores d'applications
 - ▶ Les modifications et mises à jour seront effectives sur chaque plateforme
 - ▶ Économie de budget
 - ▶ Idéale pour des applications de jeu en 2D ou en 3D

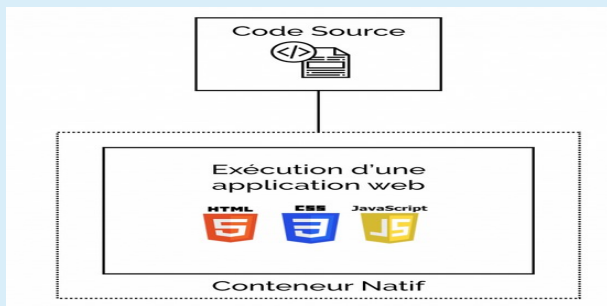
Cross-Platform

- ▶ Inconvénients
 - ▶ Problème d'accès aux APIs
 - ▶ Problème de validation des applications par les stores
 - ▶ Difficile à maintenir et à faire évoluer
 - ▶ Toutes les fonctionnalités des appareils ne peuvent pas encore être exploitées
 - ▶ Problème d'ergonomie. UX et UI non optimisées

Plateformes de développement mobile

Plateforme Hybride

- ▶ Développer dans une solution intermédiaire qui va être reconverti pour chaque environnement de manière spécifique
- ▶ Développer dans un langage tout à fait différent qui sera recompilé dans chaque plateforme



Plateforme Hybride

- ▶ Avantages
 - ▶ Développement web classique donc très bien maîtrisé par les développeurs. Utilisation de HTML5, CSS3, JavaScript et de frameworks
 - ▶ Plus de souplesse et de liberté pour le développeur. Il a en effet la possibilité de choisir le framework web qu'il souhaite

Plateformes Hybrides

- ▶ Inconvénients :
 - ▶ Faibles performances à cause de composants Javascript trop lents et trop lourds pour être utilisés dans le cadre d'un développement mobile
 - ▶ Nécessité de se connecter à internet si l'on ne veut pas tout embarquer dans l'application Toutes les fonctionnalités des appareils ne sont pas exploitables
 - ▶ Problème de validation des applications par les stores

Web mobile

- ▶ Avantages :
 - ▶ Développement web classique donc très bien maîtrisé par les développeurs.
 - ▶ Utilisation de HTML5, CSS3, JavaScript et de frameworks
 - ▶ Responsive design pour un affichage optimal et adapté aux différents supports (smartphones, tablettes, desktop)
 - ▶ Facilité de mise à jour du contenu

Web mobile

- ▶ Inconvénients
 - ▶ Nécessité d'avoir une connexion internet pour accéder au contenu
 - ▶ Pas de visibilité sur les stores d'application
 - ▶ Accessible via le navigateur.
 - ▶ Pas d'icone d'application
 - ▶ Ne s'exécute pas en plein écran